
Semantic Mapping of FWH Building Data

P23: Yujin Kim, ykim68 (<https://github.com/ykim68ncstate/ncsu-engr-ALDA-F25-Project-P23>)

1 Background

Buildings account for approximately 20% of global energy consumption and represent one of the major energy-using sectors. With the increasing deployment of sensors, actuators, and various IoT (Internet of Things) devices inside buildings, smart-building applications that integrate these components through networked platforms have expanded rapidly. These applications aim to enhance energy efficiency and optimize operational management by leveraging advances in sensing, data processing, and networking technologies. They operate not only on supervisory control and data acquisition systems (BAS/SCADA) but also on a wide range of building-level platforms, ultimately improving convenience for users and operators.

However, realizing the vision of smart buildings presents several technical challenges. One of the most critical issues is the lack of standardization in BAS operational data, as naming conventions vary widely across manufacturers and users. As a result, even buildings with similar systems exhibit different point structures, making “building-specific configuration” unavoidable for deploying applications. Consequently, control algorithms (which depend on system configuration, thermal loads, control logic, and local design) are difficult to generalize and cannot be directly transferred from one building to another.

In response, research on data standardization has gained significant momentum in the building-energy domain. The lack of interoperability has long been identified as a major barrier to scaling energy-efficiency technologies, and semantic-ontology-based standardization has emerged as a promising solution. Brick, a unified metadata schema for buildings, provides a structured ontology defining sensors, systems, and their relationships, enabling portable smart-building applications [1]. In the context of Fault Detection and Diagnosis (FDD), studies have developed software systems that automatically configure and execute applications using Brick metadata [2]. These works highlight that semantic data models are essential for achieving portable, reusable building applications.

However, even with a standardized ontology, the process of mapping real BAS data to semantic classes is not automated. BAS labels are often arbitrary, consisting of irregular strings, numeric identifiers, or site-specific naming conventions created by field engineers. To address this challenge, recent studies have attempted to classify data points based solely on the value patterns of time-series data rather than relying on label names.

The Brick by Brick 2024 Challenge hosted by AI Crowd focused on classifying irregular IoT time-series into Brick classes using only value streams, without label information [3]. CatBoost-based approaches demonstrated strong performance for such heterogeneous building IoT data, achieving a Macro F1-score of 0.616, Micro F1-score of 0.8009, accuracy of 0.9878, and recall of 0.7984 [4]. Meilan Xu et al. (2024) proposed an XGBoost-based solution that incorporated probability calibration to maintain label dependencies while suppressing extreme probabilities, successfully classifying 94 Brick subclasses from irregularly sampled data. The XGBoost model demonstrated consistent performance across evaluation sets, achieving Macro F1-scores of 0.7366–0.7391 [6].

While these studies validated the feasibility of value-stream-based automatic classification, they were tested only on curated sample datasets rather than real BAS data. No prior work has examined how such models behave in operational environments characterized by noise, irregular sampling, naming inconsistency, and missing contexts.

Therefore, this project applies value-based labeling techniques to real BAS operational data collected from an actual building, aiming to evaluate practical applicability, robustness, and stability. The objective is to verify whether functional variable types can be inferred solely from time-series patterns in real environments and to assess the feasibility of automated mapping for large-scale or multi-building deployment. Through this investigation, we aim to establish a foundation for scalable, reusable, and automated data-mapping technologies that support semantic interoperability in smart-building applications.

2 Proposed method

This project implements an automatic classification system that identifies the functional type of variables collected from a Building Automation System (BAS) based on time-series data. In practice, BAS datasets vary significantly in file structure and variable naming conventions across manufacturers and operators, and even a single piece of equipment may have its time-series data distributed across multiple files. To transform such heterogeneous data into an analyzable format, we established a pipeline that includes data structure standardization, temporal alignment, feature extraction, and model training.

2.1 Data Normalization and Preprocessing

The collected dataset consists of multiple Excel files, each containing time-series measurements for a single BAS variable. We first normalized variable names by extracting equipment and variable identifiers using regular expressions and removing unnecessary elements such as dates or sampling intervals. For example, AHU-2_SaTmp_15min_Jan and AHU-2_SaTmp_COV_Feb were both standardized to AHU-2_SaTmp. All files that mapped to the same standardized variable name were grouped together, and a unified time-series was constructed for each variable.

2.2 Feature Extraction

To enable machine learning models to recognize the temporal characteristics of each variable, we extracted a set of statistical and behavioral features:

- **Basic statistical features:** mean, standard deviation, minimum/maximum values, and range
- **Ratio-based features:** zero-value ratio, missing-value ratio
- **Signal-type features:** binary-like behavior, on/off ratio, switching count
- **Periodicity features:** daily and weekly indicators
- **Variability and pattern-based features:** signal stability and frequency of abrupt changes

Since BAS data often lack physical units, these features were designed to distinguish functional types solely from value streams, without relying on metadata such as names or units.

2.3 Automatic Label Generation and Training Dataset

Because BAS variable names are highly inconsistent, generating perfectly accurate labels is difficult. Therefore, initial labels were automatically generated by parsing substrings in variable names. Although these name-based labels are not fully reliable, they serve as large-scale preliminary annotations for model training. When golden labels (manually validated labels) were available, they were merged with the name-based labels to construct the final training labels.

2.4 XGBoost-Based Multi-Label Classification Model

This project applies an XGBoost-based multi-label classification framework to automatically identify key functional types in BAS data, such as Temperature, Flow, Pressure, and Status. The BAS time-series data used in this study exhibit highly irregular variable naming conventions, inconsistent measurement units and operating behaviors across sensors and equipment, and severe class imbalance. Under such conditions, deep learning-based sequence models tend to require large, clean datasets and often suffer from instability. In contrast, gradient boosting models are more suitable because

they train reliably even with limited labeled data and provide strong interpretability. In particular, XGBoost offers fast training performance for large real-world datasets and incorporates powerful regularization mechanisms, making it well aligned with the goals of this study.

XGBoost operates on the principle of Gradient Boosting Decision Trees (GBDT). The model first generates initial predictions on the entire dataset and calculates residuals, representing the difference between predicted and actual values. A new tree is then trained to minimize these residuals, and subsequent trees continue to reduce the remaining error. Through this iterative process, the model is progressively refined, and the final prediction is obtained by taking a weighted sum of all trees. This residual-learning approach enables XGBoost to effectively capture complex and nonlinear temporal patterns inherent in BAS time-series data. (Reference: XGBoost official documentation [5])

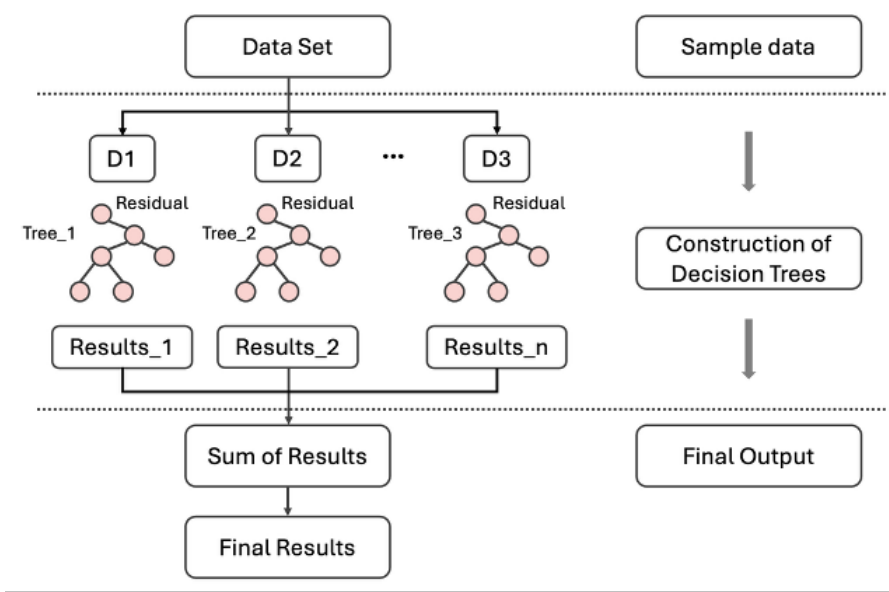


Figure 1: XGBoost model structure.

XGBoost primarily adopts a tree booster approach, where each decision tree is designed to output predictions at the leaf nodes. The training process incorporates two forms of regularization (L1 and L2): L1 regularization suppresses unnecessary splits to reduce model complexity, while L2 regularization prevents leaf weights from becoming excessively large, thereby mitigating overfitting. These regularization mechanisms are particularly important for BAS datasets, which contain substantial noise and irregular signal behavior. They help the model avoid overreacting to abrupt fluctuations or focusing excessively on a small subset of variables, ultimately enabling more generalized and stable predictions.

Because XGBoost does not inherently support multi-output classification, this project adopts a multi-label structure in which independent binary classifiers are trained for each functional type (Temperature, Flow, Pressure, Status). Each classifier produces a probability indicating whether a variable belongs to its corresponding class, and the final label is selected by choosing the class with the highest predicted probability across all classifiers. When all predicted probabilities fall below 0.5, the variable is assigned the label *none*. This architecture is well suited to BAS datasets, where class imbalance is severe (for example, Flow variables have very few samples, whereas Temperature variables appear frequently) making a single multiclass model unstable. Independent binary classifiers allow each type to be trained under its own distribution. Moreover, BAS signals occasionally exhibit overlapping functional characteristics (e.g., a variable reflecting both thermal behavior and equipment status), and this non-mutually-exclusive behavior is naturally supported by the multi-label structure. It also allows threshold values to be tuned individually for each class, enabling finer control of precision–recall trade-offs.

XGBoost further enhances interpretability by providing feature importance scores. For the statistical and behavioral features used in this study (such as mean, standard deviation, variability, and zero-

value ratio) the model computes importance based on split gain. These importance scores highlight which patterns most strongly distinguish Temperature behavior or which temporal dynamics are key in predicting Status. This interpretability is beneficial in real building environments, as it increases operator trust and provides transparent reasoning behind model outputs.

BAS datasets present several challenges: irregular sampling intervals across sensors, high label noise, strong class imbalance, and the presence of variables whose signal patterns closely resemble one another. Under these complex conditions, XGBoost’s tree-splitting mechanism effectively separates irregular patterns, and the model learns robustly even with only a small amount of golden-label data. Compared to deep sequence models such as LSTM, XGBoost exhibits lower susceptibility to overfitting and trains significantly faster, allowing efficient iteration and parameter refinement. For these reasons, XGBoost was selected as the core algorithm for the value-based functional classification model in this project.

3 Plan and Experiment

This study is based on the hypothesis that “the functional types of BAS variables can be accurately distinguished using only time-series features.” The goal is to determine whether the temporal patterns of real BAS data, where name-based labeling is impossible or highly irregular, contain sufficient information to infer the functional meaning of each variable. Because existing automated labeling studies did not provide models applicable to real-world BAS datasets, we first constructed an LSTM baseline model, which is widely used for HVAC control applications, and then evaluated its performance against the proposed XGBoost-based classifier to assess the effectiveness of the value-based labeling approach.

3.1 Dataset Description

The dataset used in this study consists of real BAS operational data collected from January 1, 2025, to October 1, 2025. Each variable is provided as an individual Excel file containing either (1) 15-minute interval data or (2) change-of-value (COV) event-driven data. In many cases, both data types coexist for the same variable, reflecting the characteristics of actual BAS environments. During preprocessing, all time-series were converted to a unified 15-minute interval, and missing segments were filled using available COV data. Finally, all variables were aligned to a global timestamp to construct a unified time-series dataset suitable for large-scale variable-level analysis.

3.2 Experimental Design

In this study, an XGBoost-based multi-label classification framework was designed to accurately identify the functional types of BAS variables. BAS data exhibit highly irregular naming conventions across sensors and equipment, non-uniform temporal patterns, and severe class imbalance between majority classes such as Temperature and Status and minority classes such as Flow and Valve. These characteristics often cause a single multi-class classifier to be biased toward the majority classes, thereby degrading predictive performance on minority categories. To address this issue, the proposed system decomposes the task into six independent binary classification problems (Temperature, Flow, Pressure, Status, Command, and Mode) each learned as a separate XGBoost model. This formulation enables each functional type to be trained independently, alleviating class imbalance, and also reflects the practical nature of BAS signals, in which a variable may simultaneously exhibit multiple physical characteristics.

XGBoost is based on the Gradient Boosting Decision Tree (GBDT) algorithm, where multiple weak learners (decision trees) are trained sequentially to correct the errors of preceding learners. The model employs the following key hyperparameters:

- `n_estimators` = 300
- `max_depth` = 4
- `learning_rate` = 0.05
- `subsample` = 0.8
- `colsample_bytree` = 0.8

- `objective = 'binary:logistic'`

A depth limit of `max_depth = 4` prevents the model from fitting noise by restricting excessively complex tree structures, while a learning rate of 0.05 ensures stable, gradual updates. The use of `subsample` and `colsample_bytree` further reduces overfitting by injecting stochasticity into both sample and feature selection for each tree, thereby improving model robustness and diversity.

XGBoost incorporates both L1 (Lasso) and L2 (Ridge) regularization, controlling leaf values and penalizing overly complex splits. This is particularly important for BAS time-series data, in which abrupt sensor fluctuations and measurement noise occur frequently. Regularization prevents the model from reacting too strongly to local anomalies and helps maintain generalization performance.

The trained set of six binary classifiers independently outputs probability scores for each functional type. A post-processing decision rule is applied to derive a single final label per variable. The class with the highest predicted probability is selected as the final label; if multiple classes show similarly high probabilities, the class with the greatest probability value is chosen. When all predicted probabilities fall below 0.5, the variable is classified as *none*, indicating the absence of a dominant functional signature. This approach preserves the advantages of the multi-label architecture while providing a practical single functional type required for real-world BAS applications.

For training and evaluation, the dataset was split using `train_test_split(test_size=0.3)`, and an independent binary classifier was constructed for each functional type. Due to the severe class imbalance in BAS data, certain classes (e.g., Pressure) occasionally contained only a single label value. In such cases, the model automatically skipped training to prevent invalid learning behavior. This mechanism enhances stability and prevents errors arising from degenerate class distributions in real-world BAS datasets.

Model performance was assessed using Precision, Recall, F1-score, and Support. Given the significant class imbalance, the Macro F1-score was used as the primary indicator. Because Macro F1-score evaluates each class equally regardless of its frequency, it captures performance degradation in minority classes such as Flow and Pressure, thereby providing a balanced assessment of the model’s capability.

3.3 Baseline model

In this project, an LSTM-based model was implemented as a baseline to compare against the proposed value-based tagging approach. The baseline model takes a single-variable time series as input and treats each BAS point as a one-dimensional sequence. Using only this temporal pattern, the model predicts six functional tags (Temperature, Flow, Pressure, Status, Command, and Mode) within a multi-label classification framework. From the unified 15-minute interval dataset, each column is treated as an independent variable, and variables with entirely missing values are removed. For the remaining variables, missing values are filled through forward-backward interpolation, and each sequence is standardized with z-score normalization based on its mean and standard deviation. Because the sequence lengths vary across variables, a common sequence length is determined according to the shortest available series, and longer time series are truncated accordingly to ensure uniform LSTM input dimensions. The processed data are then reshaped into three-dimensional tensors suitable for LSTM layers.

Label information is generated through a name-based automatic labeling function. String patterns such as “Tmp/Temp”, “Fl”, “Press”, and “Sts/Status” are detected to assign binary labels for Temperature, Flow, Pressure, Status, Command, and Mode, forming a multi-label target vector. These input sequences and label vectors are grouped by variable and split into training and testing sets using a 70:30 ratio with `train_test_split`, while fixing the random seed (42) for reproducibility.

The LSTM baseline architecture is intentionally simple. A single LSTM layer (32 units) processes each time series and outputs the final hidden state, which is then passed through a 32-unit Dense layer with ReLU activation for nonlinear transformation. The final layer is a 6-dimensional sigmoid Dense layer that independently predicts the probability of each tag. The model is trained with the Adam optimizer and binary cross-entropy loss for up to 100 epochs, with EarlyStopping applied when the validation loss does not improve for more than 10 epochs. A batch size of 8 is used to promote stable convergence, and 20% of the training data is reserved for validation to monitor overfitting. After training, predicted probabilities above 0.5 are assigned a value of 1, and those below are assigned 0,

producing the final multi-label outputs. The baseline performance is evaluated using Micro/Macro Precision, Recall, F1-score, and Hamming Loss metrics.

4 Results

This section presents the results obtained by applying the proposed BAS variable auto-classification system to real time-series data and evaluates whether the research hypothesis is supported. The model determines a final label by comparing predicted probabilities across all tag types. Table 1 shows the resulting labels (Predicted_Tag). The Golden_Tag column represents manually assigned ground-truth labels. As shown, the model achieved consistent and accurate classification for the *Temperature* and *Status* categories. In contrast, variables belonging to the *Flow* category were frequently misclassified due to similarities in value fluctuation patterns, and *Valve* variables were assigned “none” because insufficient training samples prevented learning.

Due to the characteristics of the dataset, a significant class imbalance exists, and some classes contain too few samples for valid model training. Each functional type was trained with an independent XGBoost binary classifier, and performance was evaluated using F1-score. Multi-label metrics (Precision, Recall, Micro F1, and Macro F1) were computed to assess predictive ability across all variables. Micro metrics reflect overall prediction accuracy, whereas Macro metrics measure robustness against class imbalance. The support column indicates counts from the name-based auto-labeling method.

Table 1: Predicted labels and probabilities compared with golden labels.

CSV_Variable	Predicted_Tag	Predicted_Tag_Prob	Golden_Tag
AHU-2 AvgCoilTmp	temperature	0.7269	temperature
AHU-2 AvgMaTmp	temperature	0.7358	temperature
AHU-2 ChwEnTmp	temperature	0.7269	temperature
AHU-2 ChwVlvPos	none	0.3747	status
AHU-2 MaTmp1	temperature	0.7358	temperature
AHU-2 OaFl	temperature	0.7358	flow
AHU-2 OaTmp	temperature	0.7358	temperature
AHU-2 SaFanASts	status	0.5455	status
AHU-2 SaFanBSts	status	0.5455	status
AHU-2 SaFanCSts	status	0.5455	status
AHU-2 SaFanDSts	status	0.5455	status
AHU-2 SaFanESts	status	0.5455	status
AHU-2 SaFanFSts	status	0.5455	status
AHU-2 SaFl	temperature	0.7358	flow
AHU-2 SaTmp	temperature	0.7358	temperature

Table 2: Performance evaluation based on golden labels.

Class	F1-score	Precision	Recall	Support
Flow	0.00	0.00	0.00	2
None	—	—	—	—
Status	1.00	0.86	0.92	7
Temperature	0.75	1.00	0.86	6

Table 2 summarizes these evaluation results. The *Flow* class was consistently misclassified as *Temperature* due to similarities in fluctuation patterns. The *Status* class, which exhibits distinct binary behavior, was predicted with high accuracy. The *Valve* class could not be evaluated because golden labels were not provided and training samples were insufficient. Both the *Temperature* and *Status* classes demonstrated strong performance.

The overall accuracy was 0.8, and the Macro F1-score was 0.445. The relatively low Macro F1-score reflects severe class imbalance, meaning that the model performs well on major classes but poorly on minority classes.

To establish a comparison baseline, an LSTM-based model was implemented. The LSTM yielded F1-scores close to 0.0 for most categories, except for the *Status* class, which achieved an F1-score

of 0.571. The overall accuracy was 0.4, primarily because the model consistently activated only the *Status* tag and predicted all others as zero. These results highlight the limitations of a simple sequence-only architecture and emphasize the effectiveness of the XGBoost-based feature-driven classifier, which leverages diverse statistical and behavioral features extracted from time-series data to distinguish functional characteristics more reliably.

5 Conclusion

This project aimed to address the lack of standardization in large-scale time-series data collected from Building Automation Systems (BAS) by developing a value-based mapping model that classifies functional variable types using only data patterns, without relying on variable names. To this end, we constructed a unified 15-minute interval time-series table, extracted statistical and behavioral features for each variable, and applied an XGBoost-based multi-binary classification structure to predict functional types such as Temperature, Flow, Status, and Command.

Experimental results demonstrate that the model achieved high classification accuracy for the *Status* and *Temperature* categories, reaching approximately 80% single-label accuracy and stable macro-level performance based on golden labels. This confirms that major functional types can be inferred solely from time-series patterns, even when variable names are unclear or inconsistent, highlighting the feasibility of semantic ontology-driven automatic mapping. However, minority classes such as *Flow* exhibited low performance ($F1 = 0$) due to limited training samples and strong similarity to temperature-like patterns. These limitations stem from inherent structural imbalance and the absence of physical unit information in BAS data. Future improvements will require additional sample collection, class-specific threshold adjustments, and integration with semantic ontology information such as equipment types and relational structures.

In future work, we aim to enhance classification accuracy by combining value-based approaches with name-based heuristics and apply the model to classify and map BAS data from NC State’s Centennial Campus. Furthermore, to increase model generalizability, we plan to continuously refine the classifier through model calibration, incorporation of additional data, and iterative model improvements, ultimately advancing the completeness and robustness of data stream-based classification methods.

References

- [1] Bharathan Balaji, Arka Bhattacharya, Gabriel Fierro, Jingkun Gao, Joshua Gluck, Dezhi Hong, Aslak Johansen, Jason Koh, Joern Ploennigs, Yuvraj Agarwal, Mario Bergés, David Culler, Rajesh K. Gupta, Mikkel Baun Kjærgaard, Mani Srivastava, and Kamin Whitehouse. Brick: Metadata schema for portable smart building applications. *Applied Energy*, 226:1273–1292, 2018.
- [2] Dimitris Mavrokapnidis, Gabe Fierro, Ivan Korolija, and Dimitrios Rovas. A programming model for portable fault detection and diagnosis. In *Proceedings of the 14th ACM International Conference on Future Energy Systems (e-Energy ’23)*, pages 127–131, New York, NY, USA, 2023. ACM.
- [3] Arian Prabowo, Xiachong Lin, Imran Razzak, Hao Xue, Matthew Amos, Stephen D. White, and Flora D. Salim. Brick-by-brick: Cyber-physical building data classification challenge. In *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion ’25)*, New York, NY, USA, 2025. ACM.
- [4] Chengfeng Qiu, Jiahui Zhou, Yongfeng Liao, Zhengliang Cui, and Dan Li. A unified catboost framework for iot data classification using multi-level feature extraction. In *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion ’25)*, New York, NY, USA, 2025. ACM.
- [5] XGBoost Developers. Xgboost documentation (version 2.1.3). <https://xgboost.readthedocs.io/en/stable/>. Accessed: 2025-02-27.
- [6] Meilan Xu, Zheng Wen, Xiuqing Xiang, Wenjie Fu, and Bing Wang. Winning brick by brick with daily slices: A 94-task unified xgboost solution for brick schema classification. In *Companion*

Additional References

References

- [1] Bharathan Balaji, Arka Bhattacharya, Gabriel Fierro, Jingkun Gao, Joshua Gluck, Dezhi Hong, Aslak Johansen, Jason Koh, Joern Ploennigs, Yuvraj Agarwal, Mario Bergés, David Culler, Rajesh K. Gupta, Mikkel Baun Kjærgaard, Mani Srivastava, and Kamin Whitehouse. Brick: Metadata schema for portable smart building applications. *Applied Energy*, 226:1273–1292, 2018.
- [2] Dimitris Mavrokapnidis, Gabe Fierro, Ivan Korolija, and Dimitrios Rovas. A programming model for portable fault detection and diagnosis. In *Proceedings of the 14th ACM International Conference on Future Energy Systems (e-Energy '23)*, pages 127–131, New York, NY, USA, 2023. ACM.
- [3] Arian Prabowo, Xiachong Lin, Imran Razzak, Hao Xue, Matthew Amos, Stephen D. White, and Flora D. Salim. Brick-by-brick: Cyber-physical building data classification challenge. In *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion '25)*, New York, NY, USA, 2025. ACM.
- [4] Chengfeng Qiu, Jiahui Zhou, Yongfeng Liao, Zhengliang Cui, and Dan Li. A unified catboost framework for iot data classification using multi-level feature extraction. In *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion '25)*, New York, NY, USA, 2025. ACM.
- [5] XGBoost Developers. Xgboost documentation (version 2.1.3). <https://xgboost.readthedocs.io/en/stable/>. Accessed: 2025-02-27.
- [6] Meilan Xu, Zheng Wen, Xiuqing Xiang, Wenjie Fu, and Bing Wang. Winning brick by brick with daily slices: A 94-task unified xgboost solution for brick schema classification. In *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion '25)*, New York, NY, USA, 2025. ACM.